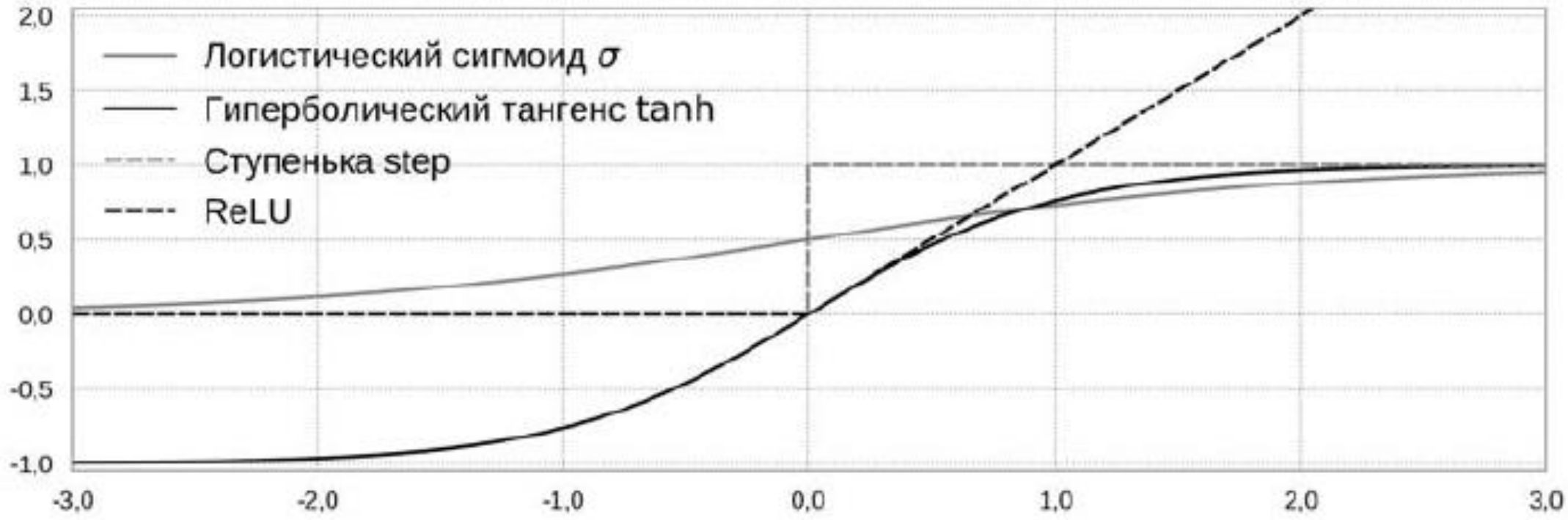
Обучение искусственных нейронных сетей

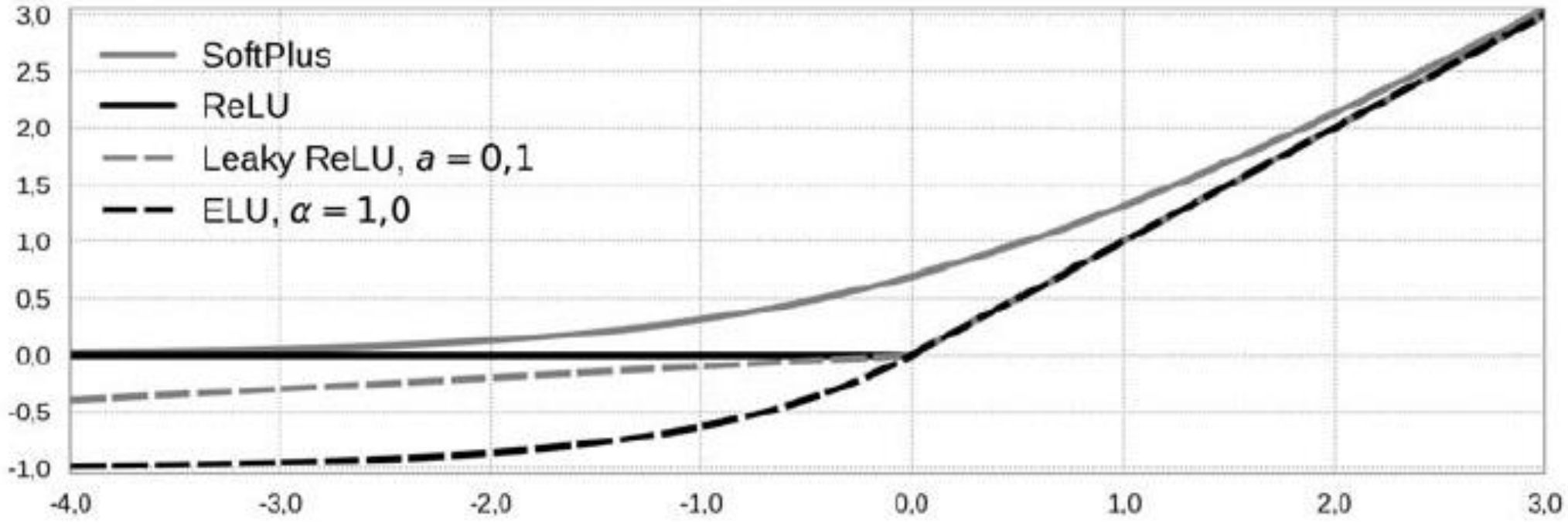
Современные функции активации в ИНС

Название функции	Формула $f(x)$	Производная $f'(x)$
Логистический сигмоид σ	$\frac{1}{1+e^{-x}}$	$f(x)\,(1-f(x))$
Гиперболический тангенс tanh	$\frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$	$1-f^{2}(x)$
SoftSign	$\frac{x}{1+\lvert x\rvert}$	$\frac{1}{(1+\lvert x\rvert)^{2}}$
Ступенька (функция Хевисайда)	$\begin{cases}0, & x<0\\ 1, & x\geq 0\end{cases}$	0
SoftPlus	$\log\left(1+e^{x}\right)$	$\frac{1}{1+e^{-x}}$
ReLU	$\begin{cases}0, & x<0\\ x, & x\geq 0\end{cases}$	$\begin{cases}0, & x<0\\ 1, & x\geq 0\end{cases}$
Leaky ReLU, Parameterized ReLU	$\begin{cases}ax, & x<0\\ x, & x\geq 0\end{cases}$	$\begin{cases}a, & x<0\\ 1, & x\geq 0\end{cases}$
ELU	$\begin{cases}\alpha\left(e^{x}-1\right), & x<0\\ x, & x\geq 0\end{cases}$	$\begin{cases}f(x)+\alpha, & x<0\\ 1, & x\geq 0\end{cases}$

Современные функции активации в ИНС



Современные функции активации в ИНС



Модификации градиентных методов

- В обычном градиентном методе шаг обновления зависел только от текущего градиента и глобального параметра скорости обучения, но никак не учитывал историю обновлений для каждого отдельного параметра.
- Если некоторые веса уже близки к своим локальным минимумам, и по этим координатам нужно двигаться медленно, а другие веса еще на середине соответствующего склона, и их можно менять гораздо быстрее.
- К тому же, само по себе регулирование скорости обучения может оказаться довольно затратным делом. Для того чтобы иметь возможность адаптировать скорость обучения для разных параметров автоматически, были созданы адаптивные методы оптимизации.
- Несмотря на то, что у них все равно есть свои собственные метапараметры, эти методы, как правило, ведут себя лучше на разреженных данных и более стабильны, чем изменение единой скорости обучения в чистом виде.

Модификации градиентных методов

- Первый из адаптивных методов оптимизации, Adagrad, основан на следующей идее: шаг изменения должен быть меньше у тех параметров, которые в большей степени варьируются в данных, и, соответственно, больше у тех, которые менее изменчивы в разных примерах (*Duchi J., Hazan E., Singer Y.* Adaptive subgradient methods for online learning and stochastic optimization // Journal of Machine Learning Research, 2011, vol. 12, no. Jul. — P. 2121–2159.).
- Поэтому Adagrad особенно полезен, когда данные сильно разрежены.
- Пусть $g_{t,i}$ градиент функции ошибки по параметру θ_i:

$$g_{t,i} = \nabla_{\theta_i} L(\theta).$$

- Тогда корректировку параметра предлагается осуществлять следующим образом:

$$\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{G_{t,ii} + \epsilon}} \cdot g_{t,i},$$

- где G_t — диагональная матрица, каждый элемент которой — это сумма квадратов градиентов соответствующего параметра за предыдущие шаги

$$G_{t,ii} = G_{t-1,ii} + g_{t,i}^2,$$

- ε – сглаживающий параметр для исключения деления на ноль

Модификации градиентных методов

- Поскольку операции ко всем координатам применяются одинаковые, приведённую формулу можно векторизовать

$$\boldsymbol{u}_t = -\frac{\eta}{\sqrt{G_{t-1} + \epsilon}} \boldsymbol{g}_{t-1}.$$

- Здесь и далее операции взятия корня и умножения продолжаются на векторы покомпонентно.

Модификации градиентных методов

- Один из плюсов Adagrad состоит в том, что о параметре скорости обучения можно больше не волноваться, так как диагональные элементы G по сути и являются индивидуальными скоростями обучения для каждого . Поэтому для параметра скорости обучения обычно используют стандартное значение 0,01, но и оно не оказывает большого влияния, так что настраивать его не обязательно.
- А главный недостаток можно заметить, если обратить внимание на то, что g^2 всегда положительно, а значит, значения G постоянно увеличиваются.
- Это приводит к слишком быстрому уменьшению скорости обучения, что плохо сказывается на обучении нейронных сетей.
- Еще один минус состоит в том, что глобальную скорость обучения в Adagrad нужно настраивать вручную, и она может оказаться хороша для одних размерностей, но плоха для других.

Модификации градиентных методов

- RMSprop— это модификация алгоритма Adagrad, основная цель которой состоит в том, чтобы попытаться исправить два описанных ранее его недостатка.
- Для этого предложено не накапливать сумму квадратов градиентов по всей истории обучения, а считать их по некоторому окну, или по всей истории, но с экспоненциально затухающими весами.
- Т.к. хранить историю предыдущих градиентов затратно, то вместо этого предложено ввести параметр инерции. Теперь у нас на каждый оптимизируемый параметр найдется свой метапараметр, и если мы обозначим этот метапараметр через ρ (ρ < 1), то изменение матрицы G можно будет записать так:

$$G_{t,ii} = \rho G_{t-1,ii} + (1-\rho) g_{t,i}^2.$$

- Дальнейшие вычисления аналогичны Adagrad

$$\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{G_{t,ii} + \epsilon}} \cdot g_{t,i},$$

$$\boldsymbol{u}_t = -\frac{\eta}{\sqrt{G_{t-1} + \epsilon}} \boldsymbol{g}_{t-1}$$

- Значение ϵ обычно берут равным 0,9, а η = 0,001.

Модификации градиентных методов

- Методу RMSprop не было посвящено ни одной специализированной статьи, равно как и не было никаких доказательств его сходимости даже для выпуклых задач.
- Все ссылки на него так или иначе приводят к курсу Джеффри Хинтона по нейронным сетям (*Tieleman T., Hinton G.* Lecture 6.5 — RMSProp: Divide the Gradient by a Running Average of its Recent Magnitude, Coursera: Neural Networks for Machine Learning, 2012. https://www:coursera:org/learn/neural-networks)

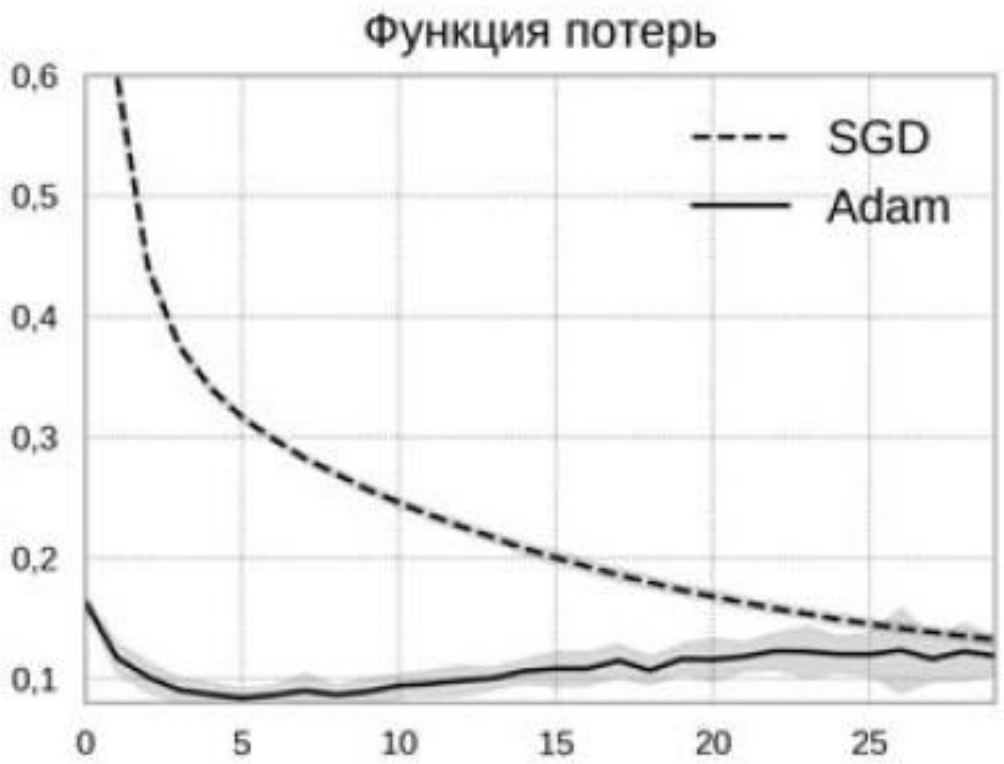
Модификации градиентных методов

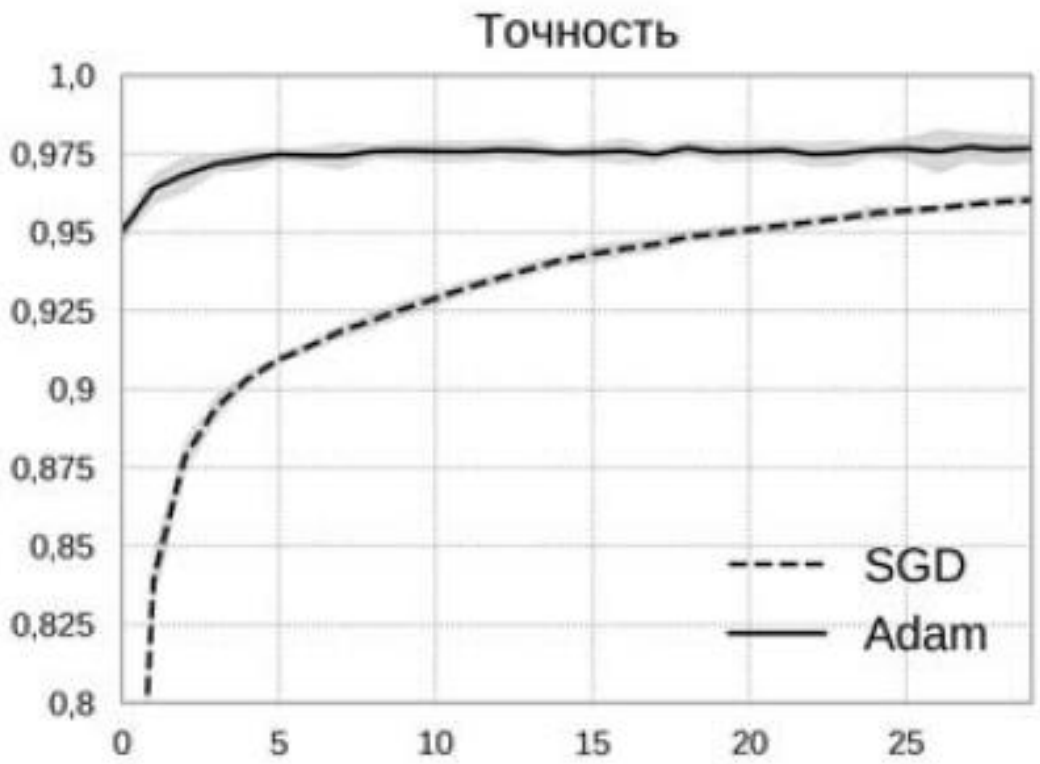
- И еще один адаптивный алгоритм оптимизации, который тоже в последнее время часто используется при обучении нейронных сетей — это Adam. Как и предыдущие два метода, он является модификацией Adagrad, но использует сглаженные версии среднего и среднеквадратичного градиентов:

$$m_t = \beta_1 m + (1 - \beta_1) g_t, \quad v_t = \beta_2 m + (1 - \beta_2) g_t^2, \quad u_t = \frac{\eta}{\sqrt{v + \epsilon}} m_t.$$

- В исходной статье с описанием метода: *Kingma D., Ba J.* Adam: A Method for Stochastic Optimization // arXiv, 2014. http://arxiv:org/abs/1412:6980 . рекомендуются значения $\beta_1 = 0{,}9$, $\beta_2 = 0{,}999$, $\epsilon = 10^{-8}$.

Модификации градиентных методов





Метод Ньютона

- Метод Ньютона основан на использовании квадратичной аппроксимации функции f в окрестности текущей точки w_k: $E(w_k + d_k) \cong E(w_k) + \nabla E(w_k)^T d_k + \frac{1}{2} {d_k}^T \nabla^2 E(w_k) d_k \to min$
- Обозначим $g_k = \nabla E(w_k)$, $H_k = \nabla^2 E(w_k)$
- Приравнивая к нулю градиент квадратичной аппроксимации, получаем:

$$\nabla_{d_k} = g_k + H_k d_k = 0 \Rightarrow H_k d_k = -g_k \Rightarrow d_k = -H_k^{-1} g_k.$$

- Таким образом, один шаг в методе Ньютона выглядит как

$$w_{k+1} = w_k - H_k^{-1} g_k$$

Анализ метода Ньютона

- Область квадратичной сходимости метода Ньютона, гарантируемая теоремой, практически совпадает с областью линейной сходимости для метода наискорейшего спуска. Поэтому с практической точки зрения разумно всегда использовать метод Ньютона вместо градиентного спуска на последнем этапе процесса обучения.
- Для корректного применения метода Ньютона необходимо выполнение двух условий:
  - В текущей точке w_k $H_k > 0$. В противном случае квадратичная аппроксимация перестает быть параболоидом с единственным минимумом. Требование $H_k > 0$ также важно для того, чтобы направление коррекции было направлением уменьшения значения функции E.
  - Квадратичная аппроксимация должна быть адекватным приближением функции E.
- Во многих случаях выполнение этих двух условий не гарантируется. В результате метод Ньютона начинает работать неустойчиво и, в ряде случаев, расходится.

Алгоритм Левенберга-Марквардта

- Учитывая вид функции ошибки, градиент и гессиан можно представить в следующем виде:

$$\frac{\partial E}{\partial w_i} = 2\sum_{n=1}^{N}(f(\boldsymbol{x}_n, \boldsymbol{w}) - y_n)\frac{\partial f(\boldsymbol{x}_n, \boldsymbol{w})}{\partial w_i},$$

$$\frac{\partial^2 E}{\partial w_j \partial w_i} = 2\sum_{n=1}^{N}\frac{\partial f(\boldsymbol{x}_n, \boldsymbol{w})}{\partial w_j}\frac{\partial f(\boldsymbol{x}_n, \boldsymbol{w})}{\partial w_i} + 2\sum_{n=1}^{N}(f(\boldsymbol{x}_n, \boldsymbol{w}) - y_n)\frac{\partial^2 f(\boldsymbol{x}_n, \boldsymbol{w})}{\partial w_j \partial w_i}.$$

- Где $o_n = f(x_n, w)$ – выход ИНС для заданного входного вектора данных x_n и векторе весовых коэффициентов w

Алгоритм Левенберга-Марквардта

- Ключевое предположение методов из семейства Левенберга-Марквардта состоит в том, что второе слагаемое в гессиане функции **E** можно отбросить. Это соответствует аппроксимации функций **f**(x_n, w) линейными функциями в окрестности текущей точки w.
- Другой аргумент в пользу отбрасывания слагаемого в гессиане состоит в том, что в окрестности оптимального решения ***f***(x_n, w) ≃ y_n.
- Тогда, с учетом отбрасывания слагаемого в гессиане, можно записать:

$$\nabla E = 2J^T(\boldsymbol{f} - \boldsymbol{t}), \quad \nabla^2 E = 2J^T J.$$

Алгоритм Левенберга-Марквардта

- Матрица $J_k^T J_k$ является неотрицательно-определенной. Поэтому для обеспечения строгой положительной определённости достаточно рассмотреть матрицу вида $J_k^T J_k + \lambda I$ для любого положительного значения .
- Метод с шагом вида

$$\boldsymbol{w}^{k+1} = \boldsymbol{w}^k - (J_k^T J_k + \lambda I)^{-1} J_k^T (\boldsymbol{f}_k - y)$$

- Где f_k – выходы ИНС для k-ого варианта весовых коэффициентов
- получил название метода Левенберга.

Алгоритм Левенберга-Марквардта

- Скорость сходимости метода наискорейшего спуска существенно зависит от выбранной шкалы измерения компонентов оптимизируемого вектора. В частности, метод Ньютона можно интерпретировать как метод наискорейшего спуска, в котором используется невырожденное преобразование координат $\mathbf{B}^{-1}$, где матрица $\mathbf{B}$ есть корень из гессиана.
- На основе этого соображения Марквардт предложил следующую модификацию метода Левенберга:

$$\boldsymbol{w}^{k+1} = \boldsymbol{w}^k - (J_k^T J_k + \lambda \operatorname{diag}(J_k^T J_k))^{-1} J_k^T (\boldsymbol{f}_k - \boldsymbol{y})$$

Алгоритм Левенберга-Марквардта

- Элементы вектора ***w*** обычно упорядочиваются сначала по слою, затем по нейронам, и, наконец, по весу каждого нейрона и его смещению.
- Значение на каждой итерации может выбираться с помощью стратегии адаптивной коррекции

- Параметр λ задаётся изначально и определяет поведение алгоритма, делая его более похожим на градиентный метод или метод Гаусса-Ньютона. В самом начале обучения, когда функция **f(x, w)**подобрана грубо, удобно использовать метод наискорейшего спуска, поэтому λ выбирается относительно большим. По мере уточнения коэффициентов w более эффективным становится метод Гаусса-Ньютона (при этом λ становится малой величиной; при λ = 0 метод вырождается в метод Гаусса-Ньютона). Так ЛМ-метод реализует адаптивную модель перехода между классами.

Алгоритм Левенберга-Марквардта

- После того, как при заданном λ вектор δ будет вычислен, необходимо принять решение о принятии модификации весов или её отклонения. Для этого необходимо рассчитать $E(w + \delta)$ и сравнить полученное значение с E(w). Если $E(w+\delta) \leq E(w)$, то необходимо уменьшить λ и изменить веса $w := w + \delta$, иначе λ увеличивается и метод применяется заново для нового λ.

Алгоритм Левенберга-Марквардта

- Для настройки величины λ часто используется вспомогательная величина , ν(обычно ν = 10). Если λ необходимо увеличить, то λ умножается на ν , иначе – делится.
- Умножение повторяется до тех пор, пока $E(w+\delta) > E(w)$. Как только выполняется неравенство $E(w+\delta) \leq E(w)$, считается, что *один обучающий цикл* (эпоха) нейросети завершился.
- Таким образом, описанный метод относятся к методам первого порядка, т.к. за счет отбрасывания второго слагаемого в гессиане удалось избавиться от необходимости вычисления второй производной функции регрессии f. При этом данные методы обладают практически квадратичной скоростью сходимости в окрестности оптимального решения.

Алгоритм Левенберга-Марквардта

- В результате процедура, реализующая обучающий цикл, имеет вид:
- 1) Весам сети присваиваются небольшие начальные значения. Рассчитывается отклик сети
- 2) Рассчитывается матрица Якоби J;
- 3) Рассчитывается градиент ошибки $g = J^T \varepsilon(w)$
- 4) Рассчитывается приближенная матрица Гессе с помощью матрицы Якоби ; $H^* = J^T J$
- 5) Решается уравнение $(H^* + \lambda I)\delta = g$ относительно неизвестного вектора δ ;
- 6) Вычисляется новое значение функции ошибки $E(w+\delta)$;
- 7) Если $E(w + \delta) > E(w)$, то $\lambda := v\,\lambda$ и перейти на шаг 5, иначе : $\lambda = \lambda / v$, $w := w + \delta$, $E(w) := E(w + \delta)$ и закончить цикл обучения